

מבנה שפות תוכנה – 2

ביטויי למבדה ופונקציות על, closure

List Comprehension

בכל התרגילים, אם לא נאמר אחרת, יש לכתוב בצורה הפונקציונלית הטובה ביותר. בתרגיל זה נתרגל שימוש בפונקציות על מובנות של python ונתיבה של פונקציות כאלו.

1. כתבו פונקציה המבטאת את הפונקציה הליניארית הבאה $Y = x/2 + 2$ באמצעות למבדה

1. עשו שימוש בביטויי הלמבדה על רשימה מ-0-10000 וצרו רשימה חדשה ע"י פונקצית על.

2. סכמו את איברי הרשימה בעזרת פונקציית על.

3. השוו בין זמן הריצה של הסכימה באמצעות פונקציית העל לזמני הריצה של סכימה בשיטה אימפרטיבית.

4. כעת, בצעו זאת באמצעות פונקציית על אחת בלבד שתסכום וגם תבצע את הקריאה ללמבדה.

2. צרו רשימה מ-1-1000 וחלקו את הרשימה לשתי רשימות שונות כך שכל המספרים הזוגיים יהיו ברשימה אחת וכל האי זוגיים ברשימה השניה, השתמשו בפונקציית על.

1. כתבו 2 פונקציות למבדה

- פונקציה שתקבל מספר ותת-רשימה מתוך רשימת המספרים הזוגיים, כפלו את תוצאת מכפלת המספרים עד למקום הנוכחי ברשימה באיבר הבא ברשימה.

- לדוגמא עבור הרשימה: 2,4,6,8,...

בצעד הראשון נכפיל $2*1$ ובשני $3*2$ ואחר כך $4*6$ וכן הלאה.

- עבור רשימת המספרים האי זוגיים, עשו שימוש בפונקציה הליניארית מלמעלה כאשר x מייצג את הסכום עד לאיבר הנוכחי $x/2 + 2 + \text{next}$ next הוא האיבר הבא ברשימה.

2. הריצו את הפונקציה המתאימה עבור כל אחת מהרשימות בעזרת פונקציית על

3. סכמו את התוצאה של כל רשימה בעזרת פונקצית על

3. מספר שלם חיובי בעל k ספרות נקרא "מספר Armstrong" אם סכום ספרותיו, כאשר כל אחת מהן מועלית בחזקת k , שווה למספר עצמו.

למשל: המספר 153 הוא בעל 3 ספרות, ומתקיים:

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

א. כתבו פונקציה טהורה בשם `is_armstrong` המקבלת מספר שלם חיובי n ומחזירה `True` אם הוא מספר Armstrong ו-`False` אחרת.

ב. כתבו פונקציה בשם `armstrong_range(n1, n2)` המחזירה את רשימת כל מספרי Armstrong בטווח שבין $n1$ ל $n2$

ג. כתבו סקריפט ראשי המקבל מספר מהמשתמש, מפעיל את הפונקציה `armstrong_range` ומדפיס את התוצאה.

אם הקלט אינו מספר שלם חיובי תקיין (למשל: מכיל אותיות או מספר שלילי), על הסקריפט להדפיס `"invalid input"`.

4. א. כתבו פונקציה המקבלת תאריך לועזי בתור מחרוזת ו2 מספרים : הראשון הוא מספר התאריכים המבוקש והשני הוא מספר הימים לדילוג, הפונקציה תחזיר רשימה של התאריכים. יש להשתמש בפונקציות על.

ב. (רשות) כתבו פונקציה דומה עבור תאריכים עבריים

5. א. כתבו פונקציה המקבלת את מעריך החזקה ומחזירה פונקציה המקבלת את בסיס החזקה ומחזירה את התוצאה.

לדוג' :

```
power_of_2 = power_function(2)
```

```
result_1 = power_of_2(3) # 3^2) 9 (צריך להחזיר
```

```
result_2 = power_of_2(5) # 5^2) 25 (צריך להחזיר
```

ב. כתבו פונקציה המקבלת מספר ומחזירה אובייקט map של פונקציות החזקה עד אותו מספר (לא כולל)

לדוג' עבור הקלט 5 יחזור אובייקט שמייצג את הרשימה : $[1, x^1, x^2, x^3, x^4]$

בנוסף, כתבו סקריפט ראשי המקבל מספר עבור החזקה הגבוהה ביותר מהמשתמש ומפעיל את הפונקציה

הסקריפט ידפיס את הטיפוס של האובייקט שהפונקציה החזירה

לאחר מכן, יקבל מספר עבור הבסיס X וידפיס tuple מתאים

לדוגמא עבור הבסיס 2 והחזקה 5 יודפס : (1, 2, 4, 8, 16)

נתון חלק מהסקריפט הראשי :

```
n = int(input("Enter number of powers:"))
```

```
result = .....
```

```
print(type(result))
```

```
base = int(input("Enter base:"))
```

....

דוגמת הרצה :

```
Enter number of powers:
5
<class 'map'>
Enter base:
2
(1, 2, 4, 8, 16)
```

ג. כתבו פונקציה המקבלת מספר x וחזקה ומחזירה את קירוב טיילור עבור e^x ע"י שימוש בסעיפים הקודמים, במימוש הפונקציה אין להשתמש בלולאות או ברשימות

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} \quad \forall x$$

תזכורת : קירוב טיילור עבור e^x מוגדר ע"י הנוסחא הבאה

כאשר סוכמים עד חזקה מסוימת.

6. מנהל משימות (closure)

כתוב פונקציה בשם `task_manager` שמאפשרת לנו לנהל משימות פשוטות באמצעות מילון. המפתח הוא שם המשימה והערך הוא מצב המשימה (`incomplete, in progress, complete`)

הפונקציה תחזיר מנהל משימות שמסוגל לבצע את הפעולות הבאות:

`add_task(task)`: הוסף משימה חדשה למילון שמייצג את המשימות. המשימה תתוסף עם מצב ברירת המחדל `"incomplete"`, אלא אם נספק מצב אחר.

`get_tasks()`: החזר את המילון שמייצג את המשימות הנוכחיות.

`complete_task(task)`: שנה את מצב המשימה להושלמה (`"complete"`).

הפונקציה של מנהל המשימות תחזיר מילון שיאפשר גישה לפונקציות

דוגמה לשימוש במנהל המשימות :

```
# Create a new task manager
tasks_manager = task_manager()

# Add tasks
tasks_manager['add_task']("Write email")
tasks_manager['add_task']("Shopping", "in progress")
```

```

tasks_manager['add_task']("Homework")

# Get the list of tasks
current_tasks = tasks_manager['get_tasks']()
print(current_tasks)
# Should print: {'Write email': 'incomplete', 'Shopping': 'in progress', 'Homework': 'incomplete'}

# Mark a task as complete
tasks_manager['complete_task']("Write email")

# Get the list of tasks after marking and deleting
current_tasks = tasks_manager['get_tasks']()
print(current_tasks)
# Should print: {'Write email': 'complete', 'Shopping': 'in progress', 'Homework': 'incomplete'}

```

7. בניית Pipeline

א. כתבו שלוש פונקציות טהורות ורגילות לעיבוד טקסט:

1. `clean_spaces(text)`: מקבלת מחרוזת ומחזירה אותה ללא רווחים בקצוות (`strip`).
2. `capitalize_text(text)`: מקבלת מחרוזת ומחזירה אותה כשאותה הראשונה בכל מילה היא אות גדולה (`title`).
3. `add_stars(text)`: מקבלת מחרוזת ומחזירה אותה כשהיא מוקפת בכוכביות (למשל: `"""hello"""`).

ב. כתבו את הפונקציות הבאות המנהלות את ה-Pipeline:

1. `create_pipeline()`: מחזירה את פונקציית הבסיס של הצנרת (פונקציית זהות המחזירה את הקלט כפי שהוא).
2. `add_to_pipeline(pipeline_fn, new_fn)`: מקבלת פונקציית צנרת קיימת (`pipeline_fn`) ופונקציה חדשה (`new_fn`), ומחזירה **פונקציה חדשה** המבצעת הרכבה של שתיהן.
הערה: אין להשתמש בלולאות (`for/while`) או ברקורסיה.

ג. כתבו **סקריפט ראשי** המבצע את השלבים הבאים:

1. יוצר `pipeline` חדש בעזרת `create_pipeline`.
2. מוסיף ל-`pipeline` את שלוש הפונקציות מסעיף א' לפי הסדר (באמצעות הקריאה ל-`add_to_pipeline`).
3. מקבל מחרוזת מהמשתמש, מפעיל עליה את ה-`pipeline` המורכב שהתקבל ומדפיס את התוצאה.
אם הקלט ריק (או מכיל רק רווחים), על הסקריפט להדפיס `"invalid input"`.

דוגמאות הרצה:

enter text:

hello functional world

Hello Functional World

enter text:

python programming

Python Programming

enter text:

invalid input